Python OOP — Interview Revision Notes

Classes, objects, the 4 pillars, magic methods and common interview questions

1. Why OOP?

- In Python, **everything is an object**: `int`, `str`, `list`, functions, even classes. `L = [1,2,3]` means L is an object of class `list`.
- That is why `L.upper()` is an error: `upper()` belongs to class `str`, not `list`. An object can only use methods defined in its class.
- **Procedural approach** gets messy in large real-world apps. OOP groups data and the functions that work on it, so code stays organised and reusable.
- **Power of OOP:** Python gives built-in data types; OOP lets you **create your own data type**. NumPy, Pandas etc. are built this way.
- Pillars / core concepts: **Class, Object, Encapsulation, Inheritance, Polymorphism, Abstraction**.

2. Class & Object

- **Class** = blueprint. It has **data** (attributes / properties / variables) and **behaviour** (methods / functions).
- **Object** = instance of a class. It follows the rules of the blueprint. `L = list()` and `s = str()` create objects the same way.
- Class names use **PascalCase**: `HumanBot`, `BankApp`.
- **Function vs Method:** a function made inside a class is a method. `len(L)` is a function; `L.append(3)` is a method.
- **Class diagram:** box with name / attributes / methods. + = public, - = private.

```
class Atm:
    def __init__(self):          # constructor
        self.pin = ''            # instance variables are created in the constructor
        self.balance = 0

obj = Atm()                      # __init__ runs automatically
obj.pin, obj.balance             # access attributes through the object
```

3. Constructor (__init__)

- A special (magic) method that runs **automatically when an object is created**. You never call it yourself.
- Used to **configure the app**: set initial values, connect to a database / vector DB, load config. Things the user shouldn't control.
- Analogy: God = programmer, Earth = class, Human = object. Birth/death setup is like the constructor; the user can't control it.
- **Parameterized** constructor: `__init__(self, a, b)`. **Non-parameterized**: `__init__(self)` only.
- The constructor can call other methods of the class with `self.func()`.

4. self

- `self` is **the object itself** (the current instance). `id(self)` == `id(obj)`.
- When you write `obj.func()`, Python actually runs `ClassName.func(obj)`, so the object is passed automatically as the first argument.
- Only an object can access the class's attributes and methods. Inside the class, one method reaches another method or variable through `self`: `self.balance`, `self.menu()`.
- Every instance method takes `self` as the first parameter. The name is a convention; the position is what matters.

5. Magic / Dunder Methods

Methods named __name__ that Python calls **automatically under certain conditions**. They let your own data type behave like a built-in one.

Method	Triggered by
`__init__(self)`	Object creation: `obj = A()`
`__str__(self)`	`print(obj)` / `str(obj)`. Must return a string. Decides how your object looks when printed
`__repr__(self)`	Developer view of the object (shell, debugging, inside lists)
`__add__(self, other)`	`obj1 + obj2`
`__sub__(self, other)`	`obj1 - obj2`

Method	Triggered by
`__mul__(self, other)`	`obj1 * obj2`
`__truediv__(self, other)`	`obj1 / obj2`
`__eq__ / __lt__ / __gt__`	==, <, >
`__len__(self)`	`len(obj)`

```
class Fraction:
    def __init__(self, n, d):
        self.num, self.den = n, d
    def __str__(self):
        return f"{self.num}/{self.den}"
    def __add__(self, other):
        return Fraction(self.num*other.den + other.num*self.den, self.den*other.den)

print(Fraction(1, 2) + Fraction(1, 3))    # 5/6
```

6. Attributes, Reference Variables & id()

- Attributes don't have to be created only inside the class. `a = A(); a.c = '3'` adds a new attribute to **that object only**, from outside.
- `A()` alone creates an object at some memory address. In `a = A()`, **a is a reference variable** that holds the address; it is not the object itself.
- `b = a` does not copy. Both names point to the same object, so a change through one is visible through the other.
- `id(x)` returns the memory identity of an object. Same id = same object.

```
p = Person("Yash", "India")
q = p                # second reference, same object
q.name = "Yasha"
print(p.name)        # Yasha  -> change via q is seen via p
```

7. Pass by Reference & Mutability

- Python passes the **object reference** to a function (called *pass by object reference*). The function can take objects as arguments and return objects.
- Objects of your own classes are **mutable**. If a function changes `person.name`, the change is visible outside and the id stays the same.
- **Mutable** (list, dict, set, custom objects): changed in place, same id. **Immutable** (int, float, str, tuple): any 'change' creates a new object with a new id.
- Can you store objects in a list / dict / tuple? Yes. In a set? **Yes**: user-defined objects are hashable by default (hash based on id), even though they are mutable. If you define `__eq__` without `__hash__`, they become unhashable.

```
def greet(person):
    print(id(person))       # same id as p outside
    person.name = "Yas"     # mutates the original object
    return person

p  = Person("Yash", "India")
p1 = greet(p)               # id(p1) == id(p), and p.name is now "Yas"
```

8. Encapsulation (Private Attributes, Getter & Setter)

- Problem: in Python, an object can read and change any attribute from outside, e.g. `obj.balance = -500`.
- Make an attribute **private** with two underscores: `self.__name`. It is hidden from `obj.__name`.
- **Name mangling:** Python renames `__name` to `_Person__name`. So `p.__name = 'x'` outside only creates a **new** attribute; the real one is untouched. `p._Person__name = 'x'` still changes it.
- **Nothing in Python is truly private**. The language trusts the developer ("we are all adults here").
- Convention: `_var` (single underscore) = internal, please don't touch. `__var` = name-mangled.
- **Getter / Setter:** public methods that read and update a private attribute. The setter can hold validation logic.
- Encapsulation = keep data private + expose it through getters and setters.

```
class Person:
    def __init__(self, name):
        self.__name = name             # private
    def get_name(self):
        return self.__name
    def set_name(self, new_name):
        if isinstance(new_name, str):  # logic before changing
            self.__name = new_name

# Pythonic version: @property
class Person2:
    def __init__(self, name): self.__name = name
    @property
    def name(self): return self.__name
    @name.setter
    def name(self, v): self.__name = v
```

9. Instance vs Static (Class) Variables & Methods

- **Instance variable**: belongs to the object; value differs per object (`self.cus_name`). Access: `obj.var`.
- **Static / class variable**: belongs to the class; one value shared by all objects (bank IFSC, a counter). Defined in the class body, outside methods. Access: `ClassName.var`.
- Make a static variable private (`__counter`) so nobody can do `Atm.counter = 'hello'`, then expose it with static getter/setter methods.
- **@staticmethod**: no `self`. Called as `Atm.get_counter()`. It can also be called through an object, but it doesn't receive the object.
- **@classmethod**: receives `cls` (the class). Used for alternate constructors / working with class state.

```
class Atm:
    __counter = 1                       # static (class) variable, private

    def __init__(self):
        self.cid = Atm.__counter        # each object gets a unique id
        Atm.__counter += 1

    @staticmethod
    def get_counter():
        return Atm.__counter

    @classmethod
    def set_counter(cls, n):
        cls.__counter = n
```

10. Class Relationships: Aggregation (has-a)

- **Aggregation** = "has a": a Customer **has an** Address. One class holds an object of another class as an attribute.
- In a class diagram, shown with a diamond on the owner side.
- The owner class **cannot access private attributes** of the other class: `self.address.__pin` fails, `self.address.city` works. Add a getter in Address and use it.
- Common pattern: keep all data private and create getters/setters for it.

```
class Address:
    def __init__(self, city, pin):
        self.city = city
        self.__pin = pin
    def get_pin(self):
        return self.__pin

class Customer:
    def __init__(self, name, address):
        self.name = name
        self.address = address          # Customer HAS-A Address

c = Customer("Yash", Address("Noida", 201305))
c.address.get_pin()
```

11. Inheritance (is-a)

- Child class gets the parent's members ("jo baap ka hai, woh beta ka hai"). Follows **DRY**: Don't Repeat Yourself.

- Example: Student and Teacher both need login and id, so put those in a parent `User` and inherit.
- **What gets inherited:** constructor, non-private attributes, non-private methods. **Private** members of the parent are **not** accessible in the child.
- A parent can't access the child's members.
- **Constructor rule:** if the child has its own `__init__`, only the child's runs and the parent's is skipped (so `self.name` won't exist). If the child has no `__init__`, the parent's runs.
- **Method overriding:** parent and child have a method with the same name → the child's version wins. Overriding `__init__` is the same idea.
- Class diagram symbol: hollow triangle arrow pointing from child to parent.

```
class User:
    def __init__(self, name):
        self.name = name
    def login(self):
        print("login")

class Student(User):
    def __init__(self, roll):
        self.roll = roll        # User.__init__ NOT called -> s.name raises AttributeError

s = Student(10)
s.login()                       # inherited method still works
```

12. super()

- `super()` calls the **parent's methods** from inside the child class, without creating a parent object.
- Main use: both parent and child have a constructor → call the parent's first with `super().__init__(...)`, then set child-only attributes.
- `super()` works only **inside** the class. `s.super().method()` outside the class is an error.
- `super()` is for **methods**. Parent instance variables live on `self`, so read them as `self.x`, not `super().x`.

```
class Phone:
    def __init__(self, price, brand, camera):
        self.price, self.brand, self.camera = price, brand, camera

class SmartPhone(Phone):
    def __init__(self, price, brand, camera, os, ram):
        super().__init__(price, brand, camera)   # parent constructor first
        self.os, self.ram = os, ram
```

13. Types of Inheritance

Type	Structure
Single	P → C. One parent, one child
Multilevel	G → P → C. C inherits data and methods of both P and G
Hierarchical	One parent, many children: Parent → Child1, Child2, Child3
Multiple	One child, many parents: `class Child(Baap, Maa)`
Hybrid	A mix of the above (e.g. hierarchical + multiple)

- **Diamond problem** (multiple inheritance): A and B both have `buy()`. Which one does C get?
- **MRO (Method Resolution Order)** decides: Python searches left to right in the parent list. `class C(A, B)` → A's `buy()`; `class C(B, A)` → B's. Check with `C.mro()` or `C.__mro__`.

14. Polymorphism

Same name, different behaviour. Three forms:

- **Method overriding**: child redefines a parent method; child gets priority (see section 11).
- **Method overloading**: same method name, different number/type of arguments. **Python doesn't support it directly**: a second `def area` replaces the first. Use **default arguments** (or `*args`) instead.
- **Operator overloading**: the same operator behaves differently by input type. `'Hello' + 'World'` concatenates, `2 + 5` adds, `[1,2,3] + [4]` merges lists. For your own class, define `__add__` etc.

```
class Geometry:
    def area(self, a, b=0):      # 'overloading' via default argument
        if b == 0:
            return 3.14 * a * a  # circle
        return a * b             # rectangle
```

15. Abstraction

- **Hide complexity**, show only what the user needs. The end user gets simple features without seeing the internals.
- **Abstract method**: a method with no logic in the abstract class. **Abstract class**: inherits ABC and has at least one `@abstractmethod`.
- You **cannot create an object** of an abstract class (TypeError).
- A child must implement **every** abstract method; only then can you create its object. This is how a parent forces children to follow a set of rules.

```
from abc import ABC, abstractmethod

class BankApp(ABC):
    @abstractmethod
    def security(self):
        pass

class MobileApp(BankApp):
    def security(self):          # must implement, else MobileApp() fails
        print("mobile security")

BankApp()      # TypeError: can't instantiate abstract class
MobileApp()    # works
```

16. Quick Interview Q&A;

Question	Answer
Class vs Object	Class = blueprint; object = instance created from it
What is self?	The current object, passed automatically as the first argument of instance methods
What is __init__?	Constructor; runs automatically on object creation to set initial state
Function vs Method	A method is a function defined inside a class and called on an object
__str__ vs __repr__	__str__ = readable output for print; __repr__ = unambiguous developer view
Is anything private in Python?	No. __var only triggers name mangling (_Class__var); still reachable
_var vs __var	_var = convention for internal use; __var = name-mangled to avoid clashes
Instance vs class variable	Instance: per object (self.x). Class: shared by all objects (Class.x)
@staticmethod vs @classmethod	Static: no self/cls, utility function. Classmethod: gets cls, can change class state / alt. constructors
Aggregation vs Inheritance	Aggregation = has-a (Customer has Address). Inheritance = is-a (Student is a User)
Child has its own __init__?	Parent's __init__ is skipped unless you call super().__init__()
What is super()?	Calls parent's method from the child; only works inside the class
What is MRO?	Order in which Python searches classes for a method (left to right); C.mro()
Overloading in Python?	Not supported directly; last definition wins. Use default args / *args
Overriding vs Overloading	Overriding = same method in child replaces parent's. Overloading = same name, different args
Can you instantiate an abstract class?	No. Need ABC + @abstractmethod; child must implement all abstract methods
Encapsulation vs Abstraction	Encapsulation hides data (private + getters/setters). Abstraction hides complexity (ABC)
b = a for objects?	No copy; both names point to the same object (same id)
Objects in a set?	Yes, they're hashable by default (id-based) unless __eq__ is defined without __hash__